Week 5 – Task 05: Comparative Analysis of CNN, RNN and LSTM for Satellite Land-Use Classification (EuroSAT)

Student name: Mohammad Javed Khan Roll number: 23MDBCS379

1. Problem and Dataset

Problem. Automatically recognising land use from satellite imagery supports forest monitoring, urban-growth tracking and infrastructure mapping. We classify 64×64 px Sentinel-2 patches into five land-cover classes and compare three deep-learning architectures – a Convolutional Neural Network (CNN), a vanilla Recurrent Neural Network (RNN) and a Long Short-Term Memory network (LSTM) – trained under identical conditions on the same data.

Dataset. EuroSAT (RGB version) – Helber, Bischke, Dengel & Borth, *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019; publicly available on Zenodo (record 7711810). The full set has 27,000 labelled images in 10 classes (2,000–3,000 each) at 64×64 px. To obtain a deliberately **small** dataset we selected five visually related classes and randomly sampled **400 images per class (2,000 in total)**.

Property	Value
Source	EuroSAT RGB, Zenodo 7711810 (Sentinel-2, 13 cities/34 countries)
Classes (5)	Forest, Pasture, River, Highway, Residential
Images per class	400 (balanced) → 2,000 total
Resolution / channels	64 × 64 px, RGB (3 channels), JPEG
Split (stratified)	Train 1,400 (70 %) · Validation 300 (15 %) · Test 301 (15 %)

Why 70/15/15. With only 2,000 images, 70 % still leaves ≈280 training images per class, 15 % validation (300) is enough for stable early stopping, and a 15 % test set (≈60 per class) gives a more reliable ROC/AUC estimate than an 80/10/10 split would (≈200 test images). Splits are stratified so every split keeps the class balance.

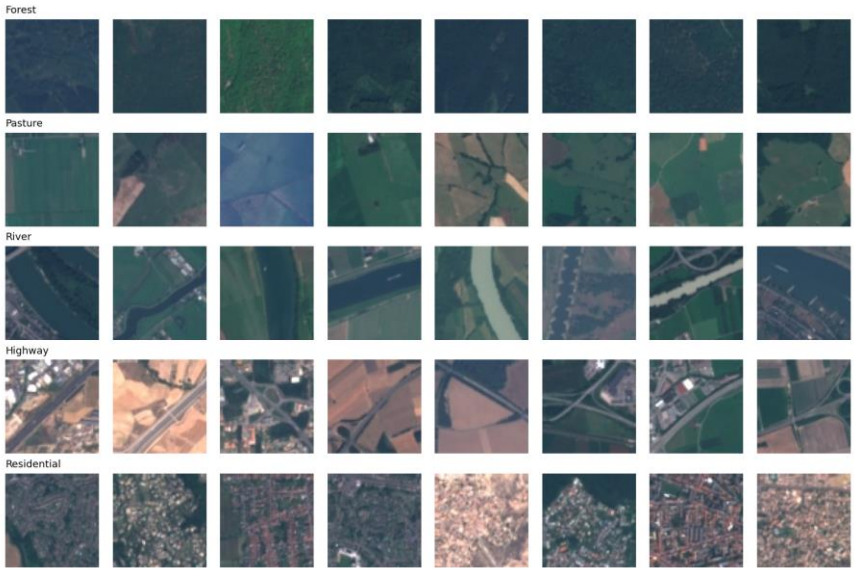


Figure 1 – Sample images (rows: Forest, Pasture, River, Highway, Residential).

2. Pre-processing

• **Rescaling and normalisation:** pixels scaled to [0,1], then standardised per channel with the *training-set* mean and standard deviation (no information from validation/test leaks in).

• **Resizing:** not required – images are already 64×64.

• **Augmentation (training only):** random horizontal flip, vertical flip and 90° rotations, applied per image. Satellite scenes have no canonical orientation, so these transforms preserve the label and effectively enlarge a small dataset, reducing over-fitting.

• **Sequence conversion for RNN/LSTM:** each image is read row by row – 64 time-steps, each a 192-dimensional vector (64 pixels × 3 channels).

3. Model Architectures

Model	Architecture	Trainable params
CNN	3 blocks of [Conv3×3–BN–ReLU]×2 + MaxPool (32→64→128 filters), global average pooling, Dropout 0.4, Linear(5)	288,549
RNN	2-layer tanh Elman RNN (hidden 128, dropout 0.3) over 64 row-steps; last hidden state → Dropout → Linear(5)	74,885
LSTM	2-layer LSTM (hidden 128, dropout 0.3) over 64 row-steps; last hidden state → Dropout → Linear(5)	297,605

Common training set-up (identical for all models): Adam (lr 1e-3, weight decay 1e-4), cross-entropy loss, batch size 64, up to 40 epochs with cosine learning-rate decay, gradient-norm clipping at 1.0, early stopping on validation loss (patience 8) with the best weights restored. Implemented in PyTorch and trained on an NVIDIA RTX 3060 Ti GPU. Every model was trained with three random seeds (42, 43, 44); figures and the main table use seed 42, and mean ± std over the three seeds is also reported.

4. Results

4.1 Comparison table (test set, seed 42)

Model	Accuracy	Precision	Recall	F1	AUC	Params	Train time (s)
CNN	0.9734	0.9747	0.9734	0.9735	0.9995	288,549	58.4
RNN	0.7309	0.7204	0.7302	0.7172	0.9290	74,885	2.4
LSTM	0.8040	0.8021	0.8037	0.8018	0.9655	297,605	3.5

Precision, recall and F1 are macro-averaged over the five classes; AUC is one-vs-rest macro-averaged.

Mean ± standard deviation over 3 seeds (robustness check):

Model	Accuracy (mean ± std)	F1 (mean ± std)	AUC (mean ± std)	Train time s (mean)
CNN	0.969 ± 0.008	0.969 ± 0.008	0.999 ± 0.000	57.9
RNN	0.738 ± 0.009	0.725 ± 0.011	0.930 ± 0.003	2.5
LSTM	0.795 ± 0.008	0.793 ± 0.009	0.957 ± 0.007	3.5

4.2 Training vs. validation curves

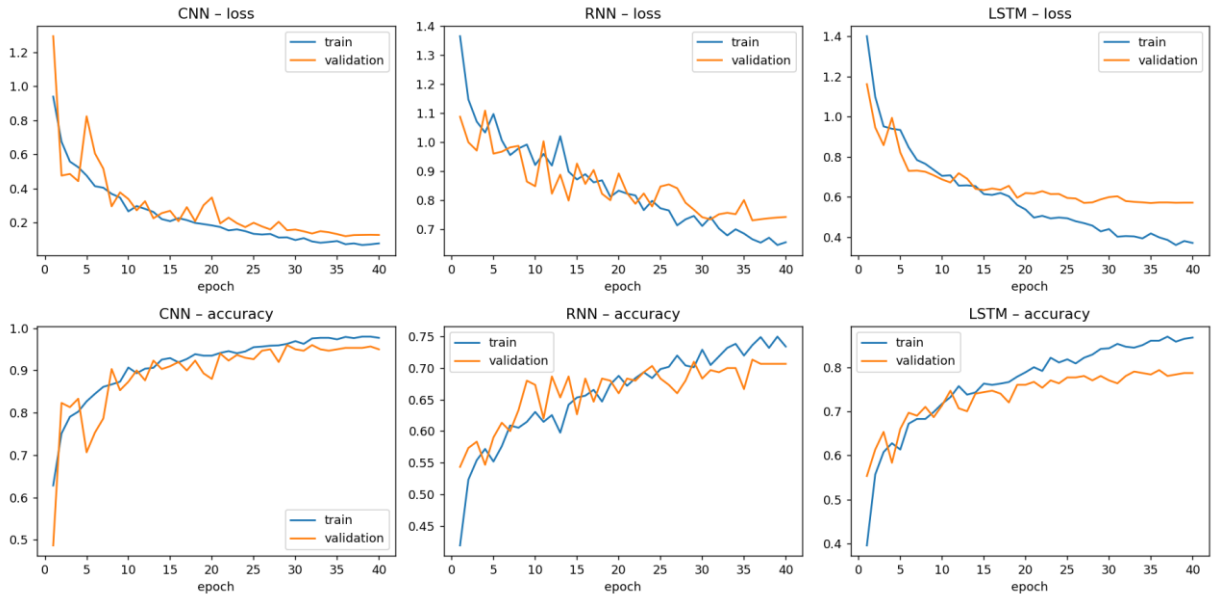


Figure 2 – Loss and accuracy per epoch. The CNN converges to low loss with a small train/validation gap; the RNN is noisy and still improving at epoch 40 (under-fitting); the LSTM shows a clear and growing train-validation gap.

4.3 Confusion matrices

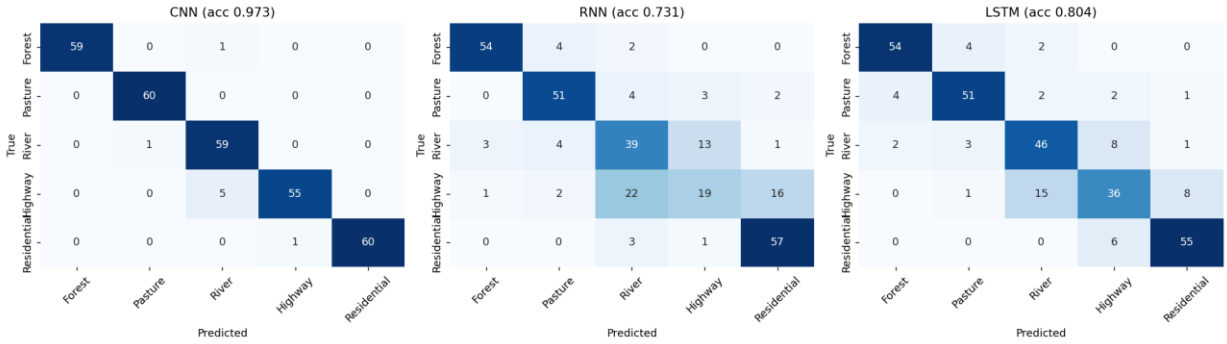


Figure 3 – Confusion matrices on the test set. Errors of the RNN/LSTM concentrate on Highway (mistaken for River or Residential) and River (mistaken for Highway).

4.4 ROC curves and AUC (one-vs-rest)

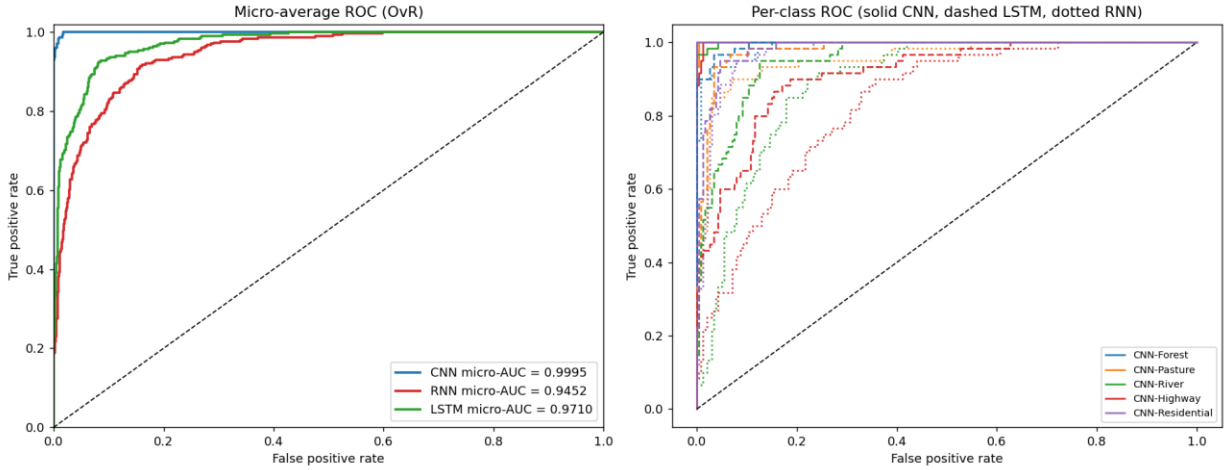


Figure 4 – Left: micro-averaged ROC of all three models on one plot. Right: per-class ROC curves.

Model	Forest	Pasture	River	Highway	Residential	Macro	Micro
CNN	1.0000	0.9997	0.9990	0.9990	1.0000	0.9995	0.9995
RNN	0.9896	0.9564	0.8925	0.8337	0.9730	0.9290	0.9452
LSTM	0.9951	0.9813	0.9537	0.9138	0.9836	0.9655	0.9710

Per-class and averaged one-vs-rest AUC on the test set.

5. Discussion

Accuracy and ranking. The CNN is clearly best (accuracy 0.973, F1 0.974, macro AUC 1.000), followed by the LSTM (0.804, 0.802, 0.966) and the plain RNN (0.731, 0.717, 0.929). The ordering CNN > LSTM > RNN is the same for every seed and the gap between models is many times larger than the seed-to-seed standard deviation, so it is not due to chance.

Why the CNN wins. (1) *Local receptive fields and weight sharing:* a 3×3 filter detects the same texture – canopy, road marking, roof pattern – anywhere in the image using very few weights. (2) *Translation invariance* through pooling: a river is a river wherever it sits in the patch. (3) *Hierarchical features:* edges → textures → object parts. The row-wise RNN/LSTM instead flattens each row into a 192-D vector, which destroys horizontal locality and the 2-D neighbourhood structure, and must push vertical information through 64 sequential steps. Note that the CNN is also the only model that exploits the 2-D structure explicitly; the RNN/LSTM have to re-discover it from scratch.

RNN vs. LSTM. The LSTM beats the plain RNN by about 7.3 accuracy points. Gates and the additive cell state mitigate vanishing gradients across the 64 steps, whereas the tanh RNN essentially relies on the last few rows and its validation curve stays noisy and unconverged. However, the LSTM has ≈4× more parameters than the RNN and its training accuracy rises well above validation accuracy (Figure 2), i.e. it starts to over-fit a 1,400-image training set.

Class-level behaviour. The visually distinctive classes (Forest, Residential) are easy for every model. The thin, linear, orientation-varying structures – Highway and River – are the hardest for the sequence models (RNN Highway F1 and AUC are the lowest in the study), because recognising a line that may run in any direction requires 2-D context. The CNN keeps AUC ≥ 0.998 for all classes. ROC curves (Figure 4) show the same story: the CNN curve hugs the top-left corner, the LSTM lies in between and the RNN is lowest.

Cost. The CNN has a similar parameter count to the LSTM yet reaches far higher accuracy, but it is slower to train (≈58 s vs. 4 s for the LSTM and 2 s for the RNN for 40 epochs), because convolutions over a 64×64 grid are more computation-heavy than a 64-step recurrence of small matrices. The extra seconds are clearly worthwhile.

Limitations. Only five classes and 2,000 images were used; the test set has 301 images, so single-run percentages carry roughly ±1–3 % uncertainty (hence the 3-seed check). The RNN/LSTM use a simple row-wise scan; patch-wise scanning, bidirectional layers, or a CNN-feature + LSTM hybrid could narrow – but, we expect, not close – the gap. The CNN could be further improved with transfer learning.

6. Conclusion

On this small satellite land-use dataset the **CNN is the best model** on every metric (accuracy 97.3 %, macro-AUC 1.000), the **LSTM is second** (80.4 %) and the **vanilla RNN is last** (73.1 %). This confirms the theory that architectures with built-in spatial inductive bias (convolution, weight sharing, pooling) are far better suited to raw image data than sequence models, even when the latter are fed the image as a sequence of rows. RNNs/LSTMs remain valuable for genuinely sequential data (time series, text, video frames) – not for single still images.

References

[1] P. Helber, B. Bischke, A. Dengel, D. Borth, “EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification,” IEEE J-STARS, 2019. Dataset: https://zenodo.org/records/7711810

[2] S. Hochreiter, J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, 1997.